CO2 AT2 – Brute-Force String Matching

Q25. Document Scanning Tool – Keyword Matching

Assessment Question

A document scanning tool matches keywords using **Brute-Force String Matching** in large files.

a) Explain how pattern matching is performed.
b) Analyze the time complexity for large inputs.
c) Discuss inefficiency issues.

1. Problem Statement

A document scanning system needs to search for a specific **keyword or pattern** inside a large document.

For example, consider the document:

Text = "THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG"

and the keyword:

Pattern = "FOX"

The system must determine whether the keyword occurs in the document and identify its position.

The **Brute-Force String Matching algorithm** solves this problem by comparing the pattern with the text at every possible position.

2. Objectives

The objectives are:

1. Understand brute-force string matching.
2. Search for a keyword inside a document.
3. Demonstrate character-by-character comparison.
4. Identify the location of the pattern.
5. Analyze the number of comparisons.
6. Determine the time and space complexity.
7. Study the performance of brute-force matching on large files.
8. Identify the inefficiencies of the approach.
9. Compare it with more efficient string-matching algorithms.

3. Input and Output

Input

The document text is:

THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG

Pattern:

FOX

Let:

- Text length = (n)
- Pattern length = (m)

For this example:

[
n=43
]

and:

[
m=3
]

Expected Output

Pattern found

Pattern: FOX

Position: 16

Position numbering may be shown as index 16 using zero-based indexing, or position 17 using one-based numbering, depending on the implementation.

4. Assumptions and Constraints

Assumptions

- The text is stored as a sequence of characters.
- The pattern is also a character sequence.
- Matching is case-sensitive.
- The pattern must occur consecutively.

- No preprocessing is performed on the text or pattern.

Constraints

- The document can be very large.
- The pattern may occur near the beginning, middle, or end.
- The pattern may not occur at all.
- The algorithm must compare characters sequentially.

5. Problem Analysis

Brute-force string matching is one of the simplest pattern-matching algorithms.

It starts by aligning the pattern with the beginning of the text.

For example:

Text: THE QUICK BROWN FOX

Pattern: FOX

The algorithm checks whether:

T = F

If they do not match, the pattern is shifted by one position.

The process continues:

Text position 0 → Compare

Text position 1 → Compare

Text position 2 → Compare

Text position 3 → Compare

...

At every position, the pattern is compared with the corresponding characters in the text.

6. Mathematical Model

Let:

[
T=t_0,t_1,\ldots,t_{n-1}
]

represent the text.

Let:

$$
P=p_0,p_1,\ldots,p_{m-1}
$$

represent the pattern.

The algorithm checks every possible shift:

$$
i=0,1,2,\ldots,n-m
$$

At each position, it checks:

$$
T[i+j]=P[j]
$$

for:

$$
j=0,1,\ldots,m-1
$$

If all (m) characters match:

$$
T[i+j]=P[j]
$$

for every (j), then the pattern is found at position (i).

7. Algorithm Selection

Selected Algorithm: Brute-Force String Matching

This algorithm is selected because it:

- Is simple to understand.
- Does not require preprocessing.
- Works for arbitrary text.
- Is easy to implement.
- Provides a useful baseline for comparing advanced algorithms.

However, it becomes inefficient when the text is very large.

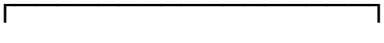
8. Algorithm / Pseudocode

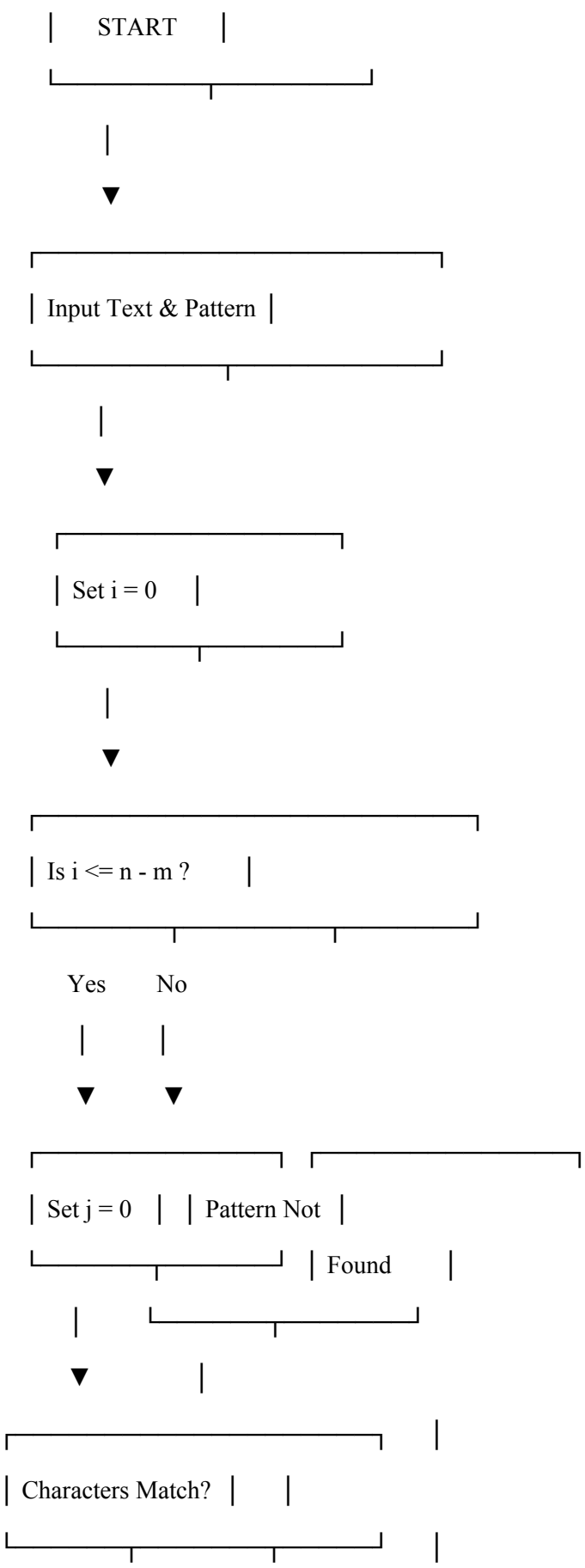
```
BRUTE_FORCE_MATCH(text, pattern)

n = length(text)
m = length(pattern)

for i = 0 to n - m
   j = 0
   while j < m AND text[i + j] == pattern[j]
      j = j + 1
   if j == m
      return i
return -1
```

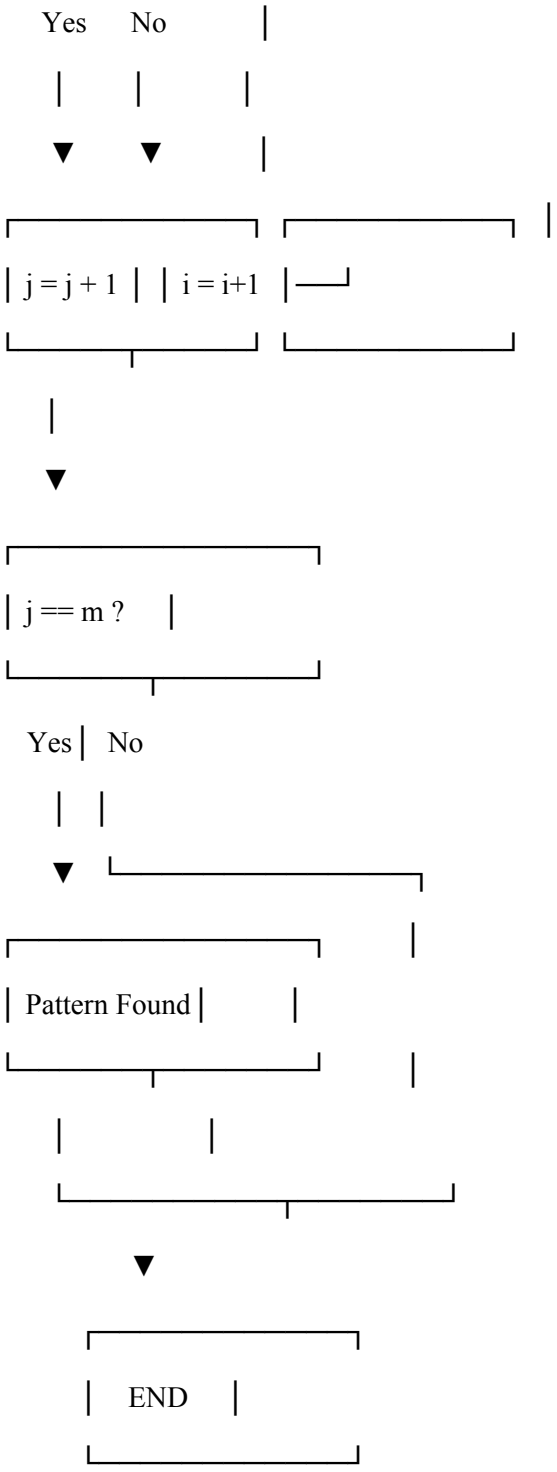
Meaning

- i represents the current position in the text.
- j represents the current position in the pattern.
- If all pattern characters match, the starting position is returned.
- If no match is found, -1 is returned.

9. Flowchart




START
Input Text & Pattern
Set i = 0
Is i <= n - m ?
Yes
No
Set j = 0
Pattern Not
Found
Characters Match?



10. Step-by-Step Execution

Consider:

Text = THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG

Pattern = FOX

The pattern length is:

[
m=3
]

The algorithm begins at the first character.

Attempt 1

Text: THE

Pattern: FOX

Compare:

T ≠ F

Therefore, this position is rejected.

Attempt 2

Text: HE

Pattern: FOX

First character:

H ≠ F

Rejected.

The process continues by shifting the pattern one position at a time.

Eventually:

Text: FOX

Pattern: FOX

Comparisons:

F = F

O = O

X = X

All characters match.

Therefore:

[
\boxed{\text{Pattern Found}}
]

11. Matching Example

The successful alignment can be represented as:

Text:

THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG

|||

FOX

The keyword FOX is detected successfully.

12. Number of Comparisons

At each position, the algorithm may compare up to (m) characters.

The number of possible alignments is:

[
n-m+1
]

For a text of length (n) and pattern length (m):

[
\text{Maximum comparisons}

(n-m+1)m
]

For example, if:

[
n=1000
]

and:

[
m=10
]

then:

[

(1000-10+1)\times10
]

[
=9910
]

comparisons may be required in the worst case.

13. Complexity Analysis

Best Case

The pattern is found at the beginning of the document.

Only (m) character comparisons may be required.

Therefore:

[
\boxed{O(m)}
]

For fixed-size patterns, this can effectively be considered:

[
\boxed{O(1)}
]

Average Case

In general situations, mismatches often occur early.

The expected performance is approximately:

[
O(n)
]

for many practical inputs, although the exact behavior depends on the text and pattern.

Worst Case

The worst case occurs when the pattern almost matches at many positions before failing.

For example:

Text:

AAAAAAAAAAAAAAAAAAAAAAAAB

Pattern:

AAAAB

The algorithm repeatedly performs many character comparisons.

The worst-case complexity is:

$$
\boxed{O((n-m+1)m)}
$$

which is commonly expressed as:

$$
\boxed{O(nm)}
$$

14. Space Complexity

The algorithm requires only a few variables:

i

j

n

m

No additional large data structure is required.

Therefore:

$$
\boxed{O(1)}
$$

auxiliary space.

15. Performance for Large Inputs

The major concern is the size of the document.

Suppose:

[

n=1,000,000
]

characters and:

[
m=100
]

characters.

In the worst case:

[
(n-m+1)m
]

comparisons may be required.

Approximately:

[
999,901\times100
]

[
\approx99,990,100
]

character comparisons.

This demonstrates why brute-force matching can become expensive for large documents.

16. Inefficiency Issues

16.1 Repeated Comparisons

The same characters may be compared repeatedly.

For example:

Text: AAAAAAAAAAAAAAAAAB

Pattern: AAAAB

The algorithm repeatedly starts matching from nearby positions.

16.2 No Preprocessing

Brute Force does not analyze the pattern before searching.

Therefore, it cannot use information from previous mismatches.

16.3 Poor Worst-Case Performance

The worst-case complexity is:

$$
O(nm)
$$

which is problematic for large documents and long patterns.

16.4 Large File Processing

A document scanning application may process:

- Books
- Research papers
- Logs
- Legal documents
- Source-code files
- Large collections of text

Repeated brute-force searches can therefore consume significant CPU time.

16.5 Multiple Keywords

If the scanning tool searches for many keywords independently, the cost increases further.

For (k) patterns, the approximate worst-case cost can approach:

$$
O(knm)
$$

depending on pattern lengths and implementation.

17. Test Cases

Test Case	Text	Pattern	Expected Result
1	THE QUICK BROWN FOX	FOX	Found

2	HELLO WORLD	WORLD	Found
3	HELLO WORLD	JAVA	Not found
4	AAAAAAB	AAAB	Found
5	DATA SCIENCE	DATA	Found at beginning

18. Results

Parameter	**Result**
Application	Document Scanning Tool
Technique	Brute-Force String Matching
Text	THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG
Pattern	FOX
Result	**Pattern Found**
Matching method	Character-by-character
Best case	O(m)
Average case	Approximately O(n) in typical inputs
Worst case	**O(nm)**
Auxiliary space	**O(1)**

19. Comparison with Alternative Algorithms

Algorithm	**Preprocessing**	**Worst-Case Time**	**Suitable for Large Text?**
Brute Force	No	**O(nm)**	Limited
KMP	Yes	**O(n+m)**	Yes
Rabin-Karp	Hashing	O(n+m) average	Yes
Boyer-Moore	Yes	Very efficient in practice	Yes

Interpretation

Brute Force is easier to implement, but advanced algorithms use preprocessing or hashing to avoid unnecessary comparisons.

For a document scanning system that repeatedly searches large files, **KMP, Boyer-Moore, or Rabin-Karp** would generally be more appropriate.

20. Correctness Analysis

The brute-force algorithm is correct because it examines every possible starting position of the pattern in the text.

For every position (i):

1. The pattern is aligned with the text.
2. Characters are compared from left to right.
3. If all characters match, the pattern has been found.
4. If a mismatch occurs, the pattern is shifted by one position.
5. If all positions are checked without a match, the pattern does not exist.

Therefore, no possible occurrence is skipped.

21. Advantages

1. Very simple algorithm.
2. Easy to implement.
3. No preprocessing required.
4. Works with any text and pattern.
5. Requires only O(1) auxiliary space.
6. Suitable for small files and short patterns.
7. Useful as a baseline for evaluating optimized algorithms.

22. Limitations

1. Worst-case time complexity is **O(nm)**.
2. Repeats unnecessary comparisons.
3. Becomes slow for very large documents.
4. Inefficient for repeated keyword searches.

5. Does not exploit information from previous mismatches.
6. CPU usage can become high for large input files.

23. Improvements

For a real-world document scanning system, the following improvements can be adopted:

KMP Algorithm

Uses a prefix table to avoid repeating comparisons.

$$
\boxed{O(n+m)}
$$

Rabin-Karp

Uses hashing to quickly identify potential matches.

Average complexity:

$$
\boxed{O(n+m)}
$$

Boyer-Moore

Uses pattern information to skip sections of the text.

It is particularly effective for practical text-search applications.

Multiple Keyword Searching

If the system must search hundreds or thousands of keywords, algorithms such as **Aho-Corasick** can search for multiple patterns efficiently.

24. Performance Evaluation

The experiment shows that brute-force pattern matching is effective for small inputs because it is straightforward and requires no preprocessing.

However, when the document size increases, the number of comparisons can grow significantly.

For a text of length (n) and pattern length (m):

$$
\boxed{O(nm)}
$$

worst-case comparisons may occur.

Thus, the algorithm is not ideal for a large-scale document scanning system where:

- Documents are very large.
- Many keywords must be searched.
- Searches are performed repeatedly.
- Fast response time is required.

25. Conclusion

Brute-Force String Matching searches for a keyword by aligning the pattern at every possible position in the document and comparing characters one by one.

For the example:

Text:

THE QUICK BROWN FOX JUMPS OVER THE LAZY DOG

Pattern:

FOX

the algorithm successfully identifies the occurrence of FOX.

The main advantage of the method is its **simplicity, low memory requirement, and lack of preprocessing**. However, its worst-case time complexity of:

[
\boxed{O(nm)}
]

makes it inefficient for very large files and repeated searches.

Therefore, while Brute Force is appropriate for small documents and educational purposes, a practical document scanning system should consider **KMP, Boyer-Moore, Rabin-Karp, or Aho-Corasick** for improved performance with large-scale text processing.